DevOps - atividade prática

Publicação de uma aplicação com Kubernetes

O que vamos fazer

Nesta atividade, vamos assumir a manutenção de uma aplicação web pequena. O código já está pronto; nosso trabalho será preparar uma entrega, colocá-la em execução e verificar se ela atende corretamente. Depois, precisaremos alterar sua configuração e sua versão sem perder o controle do que está funcionando.

A aplicação se chama *Painel de Entregas*. Ela exibe três informações: a versão publicada, uma mensagem configurada pelo grupo e o nome da instância que respondeu. Essa escolha permite observar mudanças que, em uma aplicação maior, seriam menos visíveis. Se a página mostra v2, sabemos qual entrega respondeu; se o nome do Pod muda, conseguimos acompanhar a substituição de uma instância.

O Kubernetes será responsável por organizar a execução. Vamos solicitar primeiro duas cópias da aplicação e depois três. Ao excluir uma delas, observaremos a criação de outra. Em seguida, publicaremos uma versão nova e uma versão com defeito proposital. A partir dos sinais do sistema, decidiremos como voltar à entrega que já havia sido verificada.

Por fim, executaremos uma pipeline local que reúne testes, construção da imagem, publicação e verificação HTTP. Ela reproduz tarefas que normalmente fazem parte de uma entrega de software. Nesta aula, será acionada pelo terminal, sem servidor de CI, repositório remoto ou conta externa.

Organização

Trabalhem em duplas ou trios, utilizando uma máquina por grupo. Reservem aproximadamente 100 minutos para a prática, *depois* da instalação descrita no guia. Quem estiver no Windows seguirá os blocos PowerShell; quem estiver no Ubuntu ou Arch seguirá os blocos Bash. O código e os manifestos são os mesmos.

Não é necessário decorar os comandos. O que importa é relacionar cada solicitação com o resultado observado. Por exemplo: aumentar `replicas` deve criar mais Pods; atualizar a imagem deve trocar o modelo; uma falha de readiness deve aparecer na prontidão. Se isso não ocorrer, registre o sinal e investigue antes de repetir a etapa.

Ao final, entreguem os arquivos modificados e um relato com evidências das etapas. A pasta `evidencias/` contém um modelo de relato. Registrem as observações durante a execução, sem deixar tudo para o encerramento.

1 | Conferência inicial

Entrar na pasta e testar a aplicação

A instalação de Docker, Python e Minikube está no guia. Comece com o Docker em execução e o pacote extraído. Todos os comandos seguintes partem da pasta `devops_kubernetes`, onde existe o Dockerfile. Se abrir outro terminal, volte a essa pasta antes de executar comandos.

Considerando a extração em `aulas`, conforme o guia:

Ubuntu e Arch (Bash)

```
cd ~/aulas/devops_kubernetes
pwd
mkdir -p build evidencias
chmod +x scripts/kube.sh scripts/pipeline_local.sh
docker info
minikube -p devops-k8s start --driver=docker --cpus=2 --memory=2048
./scripts/kube.sh get nodes
python3 -m unittest discover -s tests -v
```

Windows (PowerShell)

```
Set-Location "$HOME\aulas\devops_kubernetes"
Get-Location
New-Item -ItemType Directory -Force build,evidencias | Out-Null
Get-ChildItem .\scripts\*.ps1 | Unblock-File
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
docker info
minikube -p devops-k8s start --driver=docker --cpus=2 --memory=2048
.\scripts\kube.ps1 get nodes
python -m unittest discover -s tests -v
```

O nó deve chegar a `Ready`. Os oito testes devem terminar com `OK`. Eles verificam a aplicação antes de empacotá-la; não dependem de um cluster em execução. Se falharem, leia o erro e resolva-o antes de publicar.

Abra `evidencias/relato.md`, preencha os integrantes e registre as versões das ferramentas. Leia o Dockerfile e o Deployment-base. Localize a imagem, as réplicas, as probes e a label `app: entrega`. Esses elementos serão modificados ou observados nas próximas etapas.

Observe: a pasta `build/` guarda arquivos gerados durante a prática. A pasta `k8s/` contém os arquivos-base e a configuração que o grupo editará. Não são dois lugares intercambiáveis: vamos gerar o Deployment em build e aplicar esse arquivo.

2 | Primeira publicação

Construir e iniciar a versão v1

Vamos empacotar a aplicação e disponibilizar a imagem para o nó Kubernetes. O nome `entrega-local:v1` identifica a entrega; `APP_VERSION=v1` define a versão que ela mostrará na página.

Execute um bloco por vez, verificando se cada comando concluiu sem erro. O carregamento da imagem deve utilizar o mesmo perfil do cluster.

Ubuntu e Arch (Bash)

```
docker build --build-arg APP_VERSION=v1 -t entrega-local:v1 .
minikube -p devops-k8s image load entrega-local:v1
./scripts/kube.sh apply -f k8s/namespace.yaml
./scripts/kube.sh apply -f k8s/configmap.yaml
./scripts/kube.sh apply -f k8s/service.yaml
python3 scripts/render.py v1 --replicas 2 --output build/deployment.yaml
./scripts/kube.sh apply --dry-run=server -f build/deployment.yaml
./scripts/kube.sh apply -f build/deployment.yaml
./scripts/kube.sh rollout status deployment/entrega --timeout=120s
./scripts/kube.sh get pods -l app=entrega -o wide
```

Windows (PowerShell)

```
docker build --build-arg APP_VERSION=v1 -t entrega-local:v1 .
minikube -p devops-k8s image load entrega-local:v1
.\scripts\kube.ps1 apply -f k8s/namespace.yaml
.\scripts\kube.ps1 apply -f k8s/configmap.yaml
.\scripts\kube.ps1 apply -f k8s/service.yaml
python scripts/render.py v1 --replicas 2 --output build/deployment.yaml
.\scripts\kube.ps1 apply --dry-run=server -f build/deployment.yaml
.\scripts\kube.ps1 apply -f build/deployment.yaml
.\scripts\kube.ps1 rollout status deployment/entrega --timeout=120s
.\scripts\kube.ps1 get pods -l app=entrega -o wide
```

`-dry-run=server` pede à API que valide a solicitação sem salvar o Deployment. Essa conferência não testa a aplicação. O comando `apply` seguinte é que envia a configuração para publicação; `rollout status` acompanha sua conclusão.

Depois da inicialização, devem existir dois Pods com `READY 1/1`. Os nomes e endereços serão diferentes em cada execução. Capture essa saída para o relatório.

Explique por que construir a imagem no Docker não dispensou o comando `image load`. Se o Pod não iniciar, confira a tag e o perfil antes de atribuir o problema ao código.

3 | Acesso e verificação

Consultar a página e a API

Abra dois terminais na pasta do projeto. Chamaremos de **A** o terminal que manterá o encaminhamento aberto e de **B** aquele em que serão feitas as consultas.

No terminal A, execute o bloco do seu sistema e deixe-o funcionando:

Ubuntu e Arch (Bash)

```
./scripts/kube.sh port-forward service/entrega 8080:80
```

Windows (PowerShell)

```
.\scripts\kube.ps1 port-forward service/entrega 8080:80
```

No terminal B, consulte a API e execute o teste de funcionamento. No PowerShell, utilize `curl.exe` para chamar o programa curl, e não um alias do terminal.

Ubuntu e Arch (Bash)

```
curl -sS http://127.0.0.1:8080/api/status
curl -i http://127.0.0.1:8080/readyz
python3 scripts/smoke.py http://127.0.0.1:8080 --version v1 \
  --output evidencias/v1-status.json
```

Windows (PowerShell)

```
curl.exe -sS http://127.0.0.1:8080/api/status
curl.exe -i http://127.0.0.1:8080/readyz
python scripts/smoke.py http://127.0.0.1:8080 --version v1 `
  --output evidencias/v1-status.json
```

Abra `http://127.0.0.1:8080` no navegador. A versão deve ser v1, a mensagem deve ser a fornecida no ConfigMap e o nome da instância deve corresponder a um dos Pods. A API deve apresentar `ready: true`; o teste deve imprimir `SMOKE OK`. Ele também salva a resposta JSON para o relato.

Os números de porta têm funções diferentes. O navegador acessa 8080 no computador; o encaminhamento usa a porta 80 do Service; a aplicação escuta em 8000 no contêiner. Identifique esses valores nos comandos e no manifesto.

> O port-forward seleciona um Pod. Ele pode encerrar quando esse Pod for substituído ou excluído. Nas próximas etapas, pare o encaminhamento com Ctrl+C antes de uma atualização e abra-o novamente depois que ela terminar.

Capture a página. No relato, relacione a versão e o nome apresentados com a publicação feita na etapa anterior.

4 | Réplicas

Aumentar a quantidade e excluir um Pod

Agora vamos pedir três instâncias. Isso é uma mudança no estado desejado, registrada no arquivo gerado. Pare o encaminhamento do terminal A e execute no B:

Ubuntu e Arch (Bash)

```
python3 scripts/render.py v1 --replicas 3 --output build/deployment.yaml
./scripts/kube.sh apply -f build/deployment.yaml
./scripts/kube.sh rollout status deployment/entrega --timeout=120s
./scripts/kube.sh get pods -l app=entrega
```

Windows (PowerShell)

```
python scripts/render.py v1 --replicas 3 --output build/deployment.yaml
.\scripts\kube.ps1 apply -f build/deployment.yaml
.\scripts\kube.ps1 rollout status deployment/entrega --timeout=120s
.\scripts\kube.ps1 get pods -l app=entrega
```

No terminal A, acompanhe as alterações. O argumento `-w` mantém a consulta aberta e mostra mudanças.

Ubuntu e Arch (Bash)

```
./scripts/kube.sh get pods -l app=entrega -w
```

Windows (PowerShell)

```
.\scripts\kube.ps1 get pods -l app=entrega -w
```

No terminal B, exclua uma instância:

Ubuntu e Arch (Bash)

```
POD_ANTIGO=$(
  ./scripts/kube.sh get pods -l app=entrega \
    -o jsonpath='{.items[0].metadata.name}'
)
printf '%s\n' "$POD_ANTIGO" > evidencias/pod-excluido.txt
./scripts/kube.sh delete pod "$POD_ANTIGO" --wait=true
```

Windows (PowerShell)

```
$lista = .\scripts\kube.ps1 get pods -l app=entrega -o json |
  ConvertFrom-Json
$podAntigo = $lista.items[0].metadata.name
$podAntigo | Set-Content evidencias/pod-excluido.txt -Encoding utf8
.\scripts\kube.ps1 delete pod $podAntigo --wait=true
```

Acompanhe até voltar a três Pods prontos. Durante a transição, a quantidade e os estados podem variar. O nome excluído deve desaparecer e um nome novo deve surgir. Registre ambos e capture o estado final; depois encerre o watch com Ctrl+C.

O controlador criou outro Pod para manter a quantidade solicitada. Não recuperou o mesmo objeto excluído. Explique essa diferença no relato e discuta por que três Pods no mesmo nó não protegem a aplicação se a máquina inteira parar.

5 | Configuração

Alterar a mensagem sem reconstruir a imagem

Abra `k8s/configmap.yaml` no editor. Substitua apenas o valor da mensagem, preservando a estrutura do arquivo:

```
data:
  APP_MESSAGE: "Entrega configurada pelo grupo."
```

Aplique esse arquivo e consulte a API pelo port-forward de 8080. Abra novamente o encaminhamento se ele não estiver ativo.

Ubuntu e Arch (Bash)

```
./scripts/kube.sh apply -f k8s/configmap.yaml
curl -sS http://127.0.0.1:8080/api/status
```

Windows (PowerShell)

```
.\scripts\kube.ps1 apply -f k8s/configmap.yaml
curl.exe -sS http://127.0.0.1:8080/api/status
```

A mensagem dos processos já iniciados deve continuar antiga. A aplicação recebe o valor como variável de ambiente e o lê na inicialização. Atualizar o objeto ConfigMap não modifica o ambiente desses processos.

Pare o port-forward, solicite novos Pods e aguarde. A imagem continuará v1:

Ubuntu e Arch (Bash)

```
./scripts/kube.sh rollout restart deployment/entrega
./scripts/kube.sh rollout status deployment/entrega --timeout=120s
```

Windows (PowerShell)

```
.\scripts\kube.ps1 rollout restart deployment/entrega
.\scripts\kube.ps1 rollout status deployment/entrega --timeout=120s
```

Reabra o encaminhamento no terminal A. Confira no B:

Ubuntu e Arch (Bash)

```
python3 scripts/smoke.py http://127.0.0.1:8080 --version v1 \
  --message "Entrega configurada pelo grupo." \
  --output evidencias/config-status.json
```

Windows (PowerShell)

```
python scripts/smoke.py http://127.0.0.1:8080 --version v1 ‘
  --message "Entrega configurada pelo grupo." ‘
  --output evidencias/config-status.json
```

A versão deve permanecer v1 e a mensagem deve mudar. Registre por que não foi necessário construir outra imagem. A edição precisa ficar salva no arquivo local: a pipeline reaplicará esse ConfigMap.

6 | Nova versão

Publicar v2 e identificar a revisão

Vamos alterar a imagem, mantendo três réplicas e a mensagem configurada. Pare o port-forward antes de iniciar a atualização.

Ubuntu e Arch (Bash)

```
docker build --build-arg APP_VERSION=v2 -t entrega-local:v2 .
minikube -p devops-k8s image load entrega-local:v2
python3 scripts/render.py v2 --replicas 3 --output build/deployment.yaml
./scripts/kube.sh apply -f build/deployment.yaml
./scripts/kube.sh rollout status deployment/entrega --timeout=120s
./scripts/kube.sh get pods -l app=entrega
```

Windows (PowerShell)

```
docker build --build-arg APP_VERSION=v2 -t entrega-local:v2 .
minikube -p devops-k8s image load entrega-local:v2
python scripts/render.py v2 --replicas 3 --output build/deployment.yaml
.\scripts\kube.ps1 apply -f build/deployment.yaml
.\scripts\kube.ps1 rollout status deployment/entrega --timeout=120s
.\scripts\kube.ps1 get pods -l app=entrega
```

Ao final, espere três Pods prontos. Reabra o encaminhamento de 8080 no terminal A e confira no B:

Ubuntu e Arch (Bash)

```
python3 scripts/smoke.py http://127.0.0.1:8080 --version v2 \
  --message "Entrega configurada pelo grupo." \
  --output evidencias/v2-status.json
```

Windows (PowerShell)

```
python scripts/smoke.py http://127.0.0.1:8080 --version v2 `
  --message "Entrega configurada pelo grupo." `
  --output evidencias/v2-status.json
```

Precisamos guardar a revisão que acabou de ser verificada. Não adivinhe seu número: o reinício da etapa de configuração pode ter criado outra revisão. Leia a anotação do Deployment e salve-a:

Ubuntu e Arch (Bash)

```
MODELO='{{index .metadata.annotations "deployment.kubernetes.io/revision"}}'
./scripts/kube.sh get deployment entrega -o go-template="$MODELO" \
  > build/revisao-ok.txt
cat build/revisao-ok.txt
./scripts/kube.sh rollout history deployment/entrega
```

Windows (PowerShell)

```
$d = .\scripts\kube.ps1 get deployment entrega -o json |
  ConvertFrom-Json
$d.metadata.annotations.'deployment.kubernetes.io/revision' |
  Set-Content build/revisao-ok.txt -Encoding utf8
Get-Content build/revisao-ok.txt
.\scripts\kube.ps1 rollout history deployment/entrega
```

Registre o número e o resultado do teste HTTP. O número identifica a revisão do modelo dos Pods; a tag `v2` identifica a imagem. Essas duas identificações não precisam coincidir.

7 | Falha de uma entrega

Publicar a versão com defeito proposital

Na próxima entrega, o servidor continuará respondendo, mas a verificação de prontidão retornará erro. Isso permite observar uma aplicação que iniciou e, ainda assim, não deve receber tráfego. O defeito foi implementado no próprio código: a versão `quebrada` retorna 503 em `/readyz`.

Pare o port-forward e publique a imagem:

Ubuntu e Arch (Bash)

```
docker build --build-arg APP_VERSION=quebrada -t entrega-local:quebrada .
minikube -p devops-k8s image load entrega-local:quebrada
python3 scripts/render.py quebrada --replicas 3 --output build/deployment.yaml
./scripts/kube.sh apply -f build/deployment.yaml
./scripts/kube.sh rollout status deployment/entrega --timeout=30s
./scripts/kube.sh get pods -l app=entrega -o wide
./scripts/kube.sh describe deployment entrega
./scripts/kube.sh get events --sort-by=.lastTimestamp
```

Windows (PowerShell)

```
docker build --build-arg APP_VERSION=quebrada -t entrega-local:quebrada .
minikube -p devops-k8s image load entrega-local:quebrada
python scripts/render.py quebrada --replicas 3 --output build/deployment.yaml
.\scripts\kube.ps1 apply -f build/deployment.yaml
.\scripts\kube.ps1 rollout status deployment/entrega --timeout=30s
.\scripts\kube.ps1 get pods -l app=entrega -o wide
.\scripts\kube.ps1 describe deployment entrega
.\scripts\kube.ps1 get events --sort-by=.lastTimestamp
```

O acompanhamento deve terminar com erro ou timeout. Esse é o resultado previsto para a etapa; prossiga com a investigação. O prazo de 30 segundos limita apenas a espera do comando. O Deployment tem seu próprio prazo de progresso de 90 segundos, que também não realiza reversão automática.

Com recursos disponíveis, você deverá observar três Pods v2 prontos e um Pod novo não pronto. O modelo permite uma instância extra, mas não permite retirar réplicas disponíveis enquanto a substituta não fica pronta.

A coluna do Pod novo pode mostrar `Running` e `READY 0/1`. Não conclua que ele entrou em reinício contínuo: vamos conferir as duas rotas e seus registros antes de decidir.

8 | Investigar o Pod

Conferir saúde e prontidão separadamente

Identifique o Pod mais recente, que foi criado pela entrega defeituosa. No terminal B, salve sua descrição e seus logs. Depois, abra uma porta diretamente até ele:

Ubuntu e Arch (Bash)

```
POD_NOVO=$(
  ./scripts/kube.sh get pods -l app=entrega \
    --sort-by=.metadata.creationTimestamp \
    -o jsonpath='{.items[-1].metadata.name}'
)
./scripts/kube.sh describe pod "$POD_NOVO" > evidencias/incidente-describe.txt
./scripts/kube.sh logs "$POD_NOVO" > evidencias/incidente-logs.txt
./scripts/kube.sh port-forward "pod/$POD_NOVO" 8081:8000
```

Windows (PowerShell)

```
$lista = .\scripts\kube.ps1 get pods -l app=entrega -o json |
  ConvertFrom-Json
$novo = $lista.items | Sort-Object { $_.metadata.creationTimestamp } |
  Select-Object -Last 1
$podNovo = $novo.metadata.name
.\scripts\kube.ps1 describe pod $podNovo |
  Set-Content evidencias/incidente-describe.txt -Encoding utf8
.\scripts\kube.ps1 logs $podNovo |
  Set-Content evidencias/incidente-logs.txt -Encoding utf8
.\scripts\kube.ps1 port-forward "pod/$podNovo" 8081:8000
```

Deixe esse encaminhamento aberto. No terminal A, consulte as duas rotas:

Ubuntu e Arch (Bash)

```
curl -i http://127.0.0.1:8081/healthz
curl -i http://127.0.0.1:8081/readyz
```

Windows (PowerShell)

```
curl.exe -i http://127.0.0.1:8081/healthz
curl.exe -i http://127.0.0.1:8081/readyz
```

`/healthz` deve retornar 200; `/readyz`, 503. Os eventos da descrição devem mostrar a falha de readiness. A aplicação está viva, mas não pronta. A falha de readiness não pede reinício do contêiner; a verificação de liveness continua passando.

O acesso foi feito diretamente ao Pod porque queremos inspecionar justamente a instância defeituosa. Uma consulta ao Service pode alcançar uma instância saudável e não revelar o defeito que estamos analisando.

Guarde as saídas no relato e encerre o encaminhamento de 8081 com Ctrl+C. Antes de seguir, explique qual sinal justifica abandonar essa entrega.

9 | Reverter a entrega

Restaurar o modelo que funcionava

Vamos restaurar a revisão salva após a verificação de v2. Leia o número do arquivo e solicite a reversão:

Ubuntu e Arch (Bash)

```
REV_OK=$(cat build/revisao-ok.txt)
./scripts/kube.sh rollout undo deployment/entrega --to-revision="$REV_OK"
./scripts/kube.sh rollout status deployment/entrega --timeout=120s
./scripts/kube.sh get pods -l app=entrega
./scripts/kube.sh get deployment entrega \
  -o jsonpath='{.spec.template.spec.containers[0].image}'
printf '\n'
```

Windows (PowerShell)

```
$revOk = (Get-Content build/revisao-ok.txt -Raw).Trim()
.\scripts\kube.ps1 rollout undo deployment/entrega --to-revision=$revOk
.\scripts\kube.ps1 rollout status deployment/entrega --timeout=120s
.\scripts\kube.ps1 get pods -l app=entrega
$d = .\scripts\kube.ps1 get deployment entrega -o json |
  ConvertFrom-Json
$d.spec.template.spec.containers[0].image
```

A imagem deve voltar a `entrega-local:v2` e o conjunto deve estabilizar com três réplicas prontas. A mensagem continua sendo a do ConfigMap atual.

O arquivo `build/deployment.yaml` ainda descreve a versão quebrada. Se o aplicarmos novamente, pediremos outra vez aquela entrega. Corrija o arquivo gerado para representar o estado que queremos manter:

Ubuntu e Arch (Bash)

```
python3 scripts/render.py v2 --replicas 3 --output build/deployment.yaml
./scripts/kube.sh apply -f build/deployment.yaml
./scripts/kube.sh rollout status deployment/entrega --timeout=120s
```

Windows (PowerShell)

```
python scripts/render.py v2 --replicas 3 --output build/deployment.yaml
.\scripts\kube.ps1 apply -f build/deployment.yaml
.\scripts\kube.ps1 rollout status deployment/entrega --timeout=120s
```

Reabra o encaminhamento de 8080 e repita a verificação de v2. Confira a mensagem e capture a página recuperada.

Ubuntu e Arch (Bash)

```
python3 scripts/smoke.py http://127.0.0.1:8080 --version v2 \
  --message "Entrega configurada pelo grupo." \
  --output evidencias/rollback-status.json
```

Windows (PowerShell)

```
python scripts/smoke.py http://127.0.0.1:8080 --version v2 `
  --message "Entrega configurada pelo grupo." `
  --output evidencias/rollback-status.json
```

O rollback restaurou o modelo dos Pods, não todo o ambiente. Explique o que aconteceu com o ConfigMap, com a quantidade de réplicas e com os arquivos locais. Indique também qual comando realmente iniciou a reversão; o timeout da etapa anterior apenas encerrou a espera.

10 | Automatizar

Executar a pipeline local

As etapas anteriores foram feitas manualmente para observar cada efeito. Agora executaremos a sequência automatizada. Como v1 e v2 já foram utilizadas, a nova entrega será `v3`.

A implementação está em `scripts/pipeline_local.py`. Ela chama as ferramentas com seus parâmetros, acompanha os códigos de saída e para se uma etapa falhar. Os scripts Bash e PowerShell fornecidos também chamam essa implementação. Execute pelo Python para utilizar o mesmo procedimento nos dois sistemas:

Ubuntu e Arch (Bash)

```
python3 scripts/pipeline_local.py v3 3
```

Windows (PowerShell)

```
python scripts/pipeline_local.py v3 3
```

Acompanhe a saída. Primeiro devem passar os oito testes; depois virão construção, carregamento da imagem, geração e aplicação dos manifestos, acompanhamento do rollout e smoke test. A mensagem final deve indicar a conclusão de v3 com três réplicas.

A pipeline verifica se a tag já existe e recusa sobrescrevê-la. Se precisar executar outra entrega após v3, utilize v4, depois v5. Esse cuidado permite manter o pacote anterior identificável. Não apague uma imagem apenas para ignorar a recusa sem antes compreender o estado da atividade.

O script abre a porta 18080 durante sua verificação HTTP e encerra seu próprio encaminhamento ao terminar. Para mostrar a página no navegador, reabra o port-forward de 8080 após a troca dos Pods. Os registros de versão, Pods e ID da imagem são gravados em `evidencias/`.

Leia a automação

Abra `pipeline_local.py` e identifique onde são realizados o teste, a construção, a aplicação dos arquivos e a consulta HTTP. Observe que a pipeline não solicita rollback automaticamente: se falhar, ela termina com erro para que o operador examine o problema.

Os testes unitários passam mesmo tendo código para a versão quebrada porque verificam o comportamento definido para a simulação. Eles comprovam, por exemplo, que essa versão retorna 503. A readiness e a consulta HTTP verificam outra condição: se uma entrega pode atender corretamente.

Explique no relato por que esse script é uma pipeline local e o que ainda faltaria para transformá-lo em uma execução contínua acionada por alterações no código. Não é necessário implementar essa integração nesta atividade.

11 | Relato da atividade

O que entregar e explicar

Entreguem a pasta do projeto com os arquivos modificados, a pasta `evidencias/` e um relato curto, de aproximadamente duas páginas. Não enviem imagens Docker nem arquivos internos do Minikube.

As evidências devem mostrar os testes, v1 em execução, a passagem de duas para três réplicas, a substituição do Pod excluído, a mensagem alterada, a publicação v2, o problema de prontidão, a recuperação e a entrega automatizada v3. Os arquivos JSON e os registros de diagnóstico ajudam a conferir esses resultados.

Ao apresentar uma saída, indiquem o comando que a produziu e escrevam o que ela demonstra. Se o resultado divergiu, registrem o diagnóstico e a correção. Não é necessário reproduzir todos os comandos no relato; eles já estão no roteiro.

Questões para discussão

1. Qual a diferença entre uma imagem disponível no Docker do computador e uma imagem disponível para o nó Kubernetes?
2. Por que outro Pod surgiu após a exclusão? Ele tem a mesma identidade do anterior?
3. Por que o ConfigMap precisou de novos Pods para mudar a mensagem?
4. Quais sinais mostram que a entrega defeituosa estava viva, mas não pronta?
5. O que o rollback recuperou? Por que foi necessário ajustar também o arquivo gerado?
6. Que verificações a pipeline faz e quais problemas ainda poderiam ocorrer mesmo após sua conclusão?

Avaliação

Etapa	Critério e pontuação
Publicação v1	Testes e resposta da aplicação demonstrados: 2,0.
Réplicas	Escala e substituição do Pod explicadas: 1,5.
Configuração	Mensagem alterada e reinício justificado: 1,5.
Atualização v2	Entrega conferida e revisão salva: 1,0.
Falha e recuperação	Diagnóstico e rollback comprovados: 2,0.
Automação	Pipeline concluída e relato organizado: 2,0.

Como continuação opcional, publique v4 com duas réplicas pela pipeline. Confira o arquivo gerado e discuta quais valores deveriam ser parametrizados ao levar o mesmo procedimento para outro ambiente.

12 | Consulta durante a prática

Problemas frequentes

Sinal	O que conferir
`ErrImageNeverPull`	Nome e tag da imagem; carregamento no perfil devops-k8s. A política Never não baixa uma imagem ausente.
Pod `Pending`	Eventos de agendamento e recursos livres; consulte describe pod.
`Running 0/1`	Readiness, configuração e porta. Na entrega quebrada, o 503 é proposital.
Reinícios contínuos	Logs, inclusive –previous, e eventos. Esse não é o comportamento previsto da falha de readiness.
Conexão recusada	Port-forward aberto, porta livre e Pod selecionado ainda existente.
Mensagem antiga	ConfigMap aplicado e Pods recriados para ler o ambiente.
Tag já existente	Escolher a próxima versão; a pipeline preserva a identificação das anteriores.
Erro de YAML	Indentação e conteúdo do arquivo; salvar pelo render.py com –output evita UTF-16 no Windows.

Use estes comandos para consultar a situação atual. Para analisar uma instância específica, substitua `deployment/entrega` por `pod/NOME` no comando de logs.

Ubuntu e Arch (Bash)

```
./scripts/kube.sh get deployment,replicaset,pods,service
./scripts/kube.sh describe deployment entrega
./scripts/kube.sh get events --sort-by=.lastTimestamp
./scripts/kube.sh logs deployment/entrega --tail=30
```

Windows (PowerShell)

```
.\scripts\kube.ps1 get 'deployment,replicaset,pods,service'
.\scripts\kube.ps1 describe deployment entrega
.\scripts\kube.ps1 get events --sort-by=.lastTimestamp
.\scripts\kube.ps1 logs deployment/entrega --tail=30
```

No Windows, mantenha o Docker Desktop aberto. Se um script PowerShell for bloqueado, consulte a preparação de execução no guia. No Linux, confira o serviço Docker e a sessão que recebeu acesso ao grupo.

Antes de repetir a publicação, identifique se o problema está no arquivo, no agendamento, no início do processo ou na prontidão. Cada situação pede uma correção diferente. Os conceitos e suas referências estão no guia introdutório.

13 | Encerramento

Guardar os resultados e desligar o ambiente

Depois de conferir a entrega v3, guarde o estado final e organize os arquivos do relato. Interrompa os encaminhamentos de portas com Ctrl+C.

Ubuntu e Arch (Bash)

```
./scripts/kube.sh get pods -l app=entrega -o wide \
  > evidencias/final-pods.txt
./scripts/kube.sh get deployment entrega -o yaml \
  > evidencias/final-deployment.yaml
```

Windows (PowerShell)

```
.\scripts\kube.ps1 get pods -l app=entrega -o wide |
  Set-Content evidencias/final-pods.txt -Encoding utf8
.\scripts\kube.ps1 get deployment entrega -o yaml |
  Set-Content evidencias/final-deployment.yaml -Encoding utf8
```

Aqui o YAML é uma evidência. Para os manifestos que serão aplicados durante o roteiro, continue utilizando `render.py -output`, que salva o arquivo em UTF-8 de forma consistente.

Se não precisar mais da aplicação no cluster, remova apenas o namespace da atividade e pare o perfil:

Ubuntu e Arch (Bash)

```
./scripts/kube.sh delete namespace aula-k8s
minikube -p devops-k8s stop
```

Windows (PowerShell)

```
.\scripts\kube.ps1 delete namespace aula-k8s
minikube -p devops-k8s stop
```

Excluir o namespace remove seus objetos. Parar o perfil desliga o ambiente local, que poderá ser iniciado novamente. Para remover definitivamente o cluster do exercício, utilize `minikube -p devops-k8s delete`, somente depois de guardar os resultados.

Retomando o que foi feito

Partimos de um programa pronto, construímos uma imagem e descrevemos sua execução em arquivos. Verificamos a resposta, mudamos a quantidade de instâncias e observamos a reconciliação. Separar configuração e imagem permitiu alterar a mensagem sem reconstruir o pacote.

A entrega defeituosa mostrou por que iniciar um processo e disponibilizar uma aplicação são situações diferentes. A investigação levou à reversão de uma revisão conhecida. Por último, a automação reuniu essas tarefas em uma sequência repetível. Essas são operações comuns no trabalho de entrega e manutenção de aplicações, mesmo quando as ferramentas ou o tamanho do ambiente são diferentes.